# SQL alapok .NET backend fejlesztőknek

## Gyakorlatorientált tananyag • PostgreSQL • ASP.NET Core • Entity Framework Core

Cél: önállóan megérteni és megírni a mindennapi CRUD-, riport- és kapcsolt adatlekérdezéseket; felismerni a teljesítmény- és adatkonzisztencia-problémák alapjait.

Tanulási útvonal

1. Adatmodell és SELECT → 2. Szűrés és rendezés → 3. JOIN → 4. Aggregálás → 5. Módosítások → 6. Relációs integritás → 7. Tranzakciók és indexek → 8. EF Core/LINQ → 9. Gyakorlatok.

A példák PostgreSQL szintaxist használnak. A legtöbb SELECT és JOIN más relációs adatbázisban is hasonló.

# 1. Relációs alapok és mintadatbázis

A tábla sorokból (rekordokból) és oszlopokból áll. A PRIMARY KEY egyedileg azonosít egy sort; a FOREIGN KEY egy másik tábla kulcsára hivatkozik. A NULL hiányzó/ismeretlen értéket jelöl, nem nullát vagy üres szöveget.

```
CREATE TABLE users (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE
);

CREATE TABLE orders (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id),
    total NUMERIC(12,2) NOT NULL CHECK (total >= 0),
    status TEXT NOT NULL DEFAULT 'pending',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

A FOREIGN KEY miatt nem lehet olyan order rekordot beszúrni, amelynek user_id értéke nem létező felhasználóra mutat (ha a constraint aktív).

```
INSERT INTO users (name, email) VALUES
('Anna', 'anna@example.com'),
('Bela', 'bela@example.com'),
('Csaba', 'csaba@example.com');

INSERT INTO orders (user_id, total, status) VALUES
(1, 5000, 'paid'), (1, 8000, 'paid'),
(2, 3000, 'pending');
```

## SQL típusok, amelyeket ismerj

| Típus | Mire való |
|---|---|
| INTEGER / BIGINT | Egész szám, azonosító |
| TEXT / VARCHAR(n) | Szöveg |
| BOOLEAN | Igaz/hamis |
| NUMERIC(p,s) | Pontos decimális érték: pénzhez előnyös |
| DATE / TIMESTAMPTZ | Dátum / időpont időzónakezeléssel |
| UUID | Globálisan egyedi azonosító |

# 2. SELECT, WHERE, ORDER BY, LIMIT

```
SELECT id, name, email
FROM users
WHERE name ILIKE 'a%'
ORDER BY name ASC
LIMIT 10 OFFSET 0;
```

Az ILIKE PostgreSQL-specifikus, kis- és nagybetűt nem megkülönböztető mintakeresés. A % tetszőleges hosszúságú karaktersorozatot jelent. Az OFFSET alapú lapozás nagy táblán drágává válhat; mély

lapozáshoz gyakran keyset pagination jobb.

```
SELECT * FROM orders
WHERE total BETWEEN 3000 AND 8000
  AND status IN ('paid', 'pending');

SELECT * FROM orders WHERE status <> 'paid';
SELECT * FROM orders WHERE created_at IS NOT NULL;
```

NULL összehasonlításra IS NULL / IS NOT NULL való, nem = NULL. A WHERE feltétel csak a TRUE eredményű sorokat tartja meg; NULL esetén az összehasonlítás gyakran UNKNOWN.

## 3. JOIN: kapcsolt táblák lekérdezése

```
SELECT u.name, o.id, o.total
FROM users AS u
JOIN orders AS o ON o.user_id = u.id;
```

JOIN = INNER JOIN: csak a párosítható sorok maradnak. Egy user több order rekorddal több eredménysorban jelenhet meg.

```
SELECT u.name, o.id, o.total
FROM users u
LEFT JOIN orders o ON o.user_id = u.id;
```

LEFT JOIN = LEFT OUTER JOIN: minden felhasználó megmarad, rendelés nélkül a jobb oldali oszlopok NULL-ok. RIGHT JOIN a jobb oldalt, FULL JOIN mindkét oldalt őrzi meg.

```
-- Rendelés nélküli felhasználók
SELECT u.id, u.name
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE o.id IS NULL;

-- Ugyanez NOT EXISTS-szel
SELECT u.id, u.name FROM users u
WHERE NOT EXISTS (
  SELECT 1 FROM orders o WHERE o.user_id = u.id
);
```

Gyakori hiba: LEFT JOIN után WHERE o.status = 'paid' kiszűri a NULL sorokat. Ha minden user maradjon meg, a feltételt tedd az ON részbe: LEFT JOIN orders o ON ... AND o.status = 'paid'.

## 4. Aggregáló függvények és GROUP BY

| Függvény | Jelentés |
|---|---|
| COUNT(*) | Összes sor száma |
| COUNT(o.id) | Nem NULL id-k száma |
| COUNT(DISTINCT user_id) | Különböző felhasználók száma |
| SUM(total), AVG(total) | Összeg, átlag |
| MIN(total), MAX(total) | Minimum, maximum |

```
SELECT user_id,
```

```
       COUNT(*) AS order_count,
       SUM(total) AS total_spent,
       AVG(total) AS avg_order
FROM orders
GROUP BY user_id
HAVING SUM(total) >= 8000
ORDER BY total_spent DESC;
```

WHERE az aggregálás előtt sorokat szűr; HAVING a GROUP BY után csoportokat szűr. Aggregálás nélkül is lehet HAVING, de jellemzően GROUP BY-val együtt használjuk.

```
SELECT u.id, u.name,
       COUNT(o.id) AS order_count,
       COALESCE(SUM(o.total), 0) AS total_spent
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.name;
```

LEFT JOIN esetén COUNT(*) a rendelés nélküli userre is 1-et adhat, ezért COUNT(o.id) kell. SUM üres halmazon NULL; COALESCE ezt 0-ra alakítja.

## 5. DISTINCT, alkérdezések, EXISTS és CTE

```
SELECT DISTINCT user_id FROM orders;

SELECT * FROM orders
WHERE total > (SELECT AVG(total) FROM orders);

SELECT u.* FROM users u
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id);

WITH spend AS (
  SELECT user_id, SUM(total) AS amount
  FROM orders GROUP BY user_id
)
SELECT * FROM spend WHERE amount > 5000;
```

A CTE (WITH) névvel ellátott részlekérdezés: javíthatja az olvashatóságot, de nem garantálja, hogy gyorsabb lesz. EXISTS arra jó, hogy egy kapcsolódó rekord létezését ellenőrizd, anélkül hogy a JOIN miatt megsokszoroznád a sorokat.

## 6. INSERT, UPDATE, DELETE és RETURNING

```
INSERT INTO users (name, email)
VALUES ('Dora', 'dora@example.com')
RETURNING id, name;

UPDATE orders SET status = 'paid'
WHERE id = 3
RETURNING id, status;

DELETE FROM orders WHERE id = 3
RETURNING id;
```

Mindig ellenőrizd a WHERE feltételt UPDATE/DELETE előtt: nélküle minden sor módosulhat/törlődhet. PostgreSQL RETURNING segítségével az érintett rekordokat egy körben visszakapod.

### UPSERT PostgreSQL-ben

```
INSERT INTO users (name, email)
VALUES ('Anna uj neve', 'anna@example.com')
ON CONFLICT (email)
DO UPDATE SET name = EXCLUDED.name
RETURNING id;
```

Az ON CONFLICT csak megfelelő UNIQUE vagy exclusion constraint mellett működik. Az EXCLUDED a beszúrni próbált értékeket jelöli.

## 7. Constraint-ek és adatmodellezés

PRIMARY KEY: egyedi, nem NULL azonosító. FOREIGN KEY: hivatkozási integritás. UNIQUE: duplikációtiltás. NOT NULL: kötelező érték. CHECK: üzleti szabály egy részének adatbázis-szintű védelme.

```
CREATE TABLE course_enrollments (
  user_id BIGINT NOT NULL REFERENCES users(id),
  course_id BIGINT NOT NULL,
  enrolled_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, course_id)
);
```

Kapcsolatok: 1:N például user → orders; N:M például users ↔ courses, kapcsolótáblával. Normalizálás célja a redundancia és módosítási anomáliák csökkentése. Ne tárolj több order ID-t vesszővel elválasztva egy oszlopban.

### Törlési viselkedés

ON DELETE RESTRICT / NO ACTION: a hivatkozott sor törlését korlátozza. ON DELETE CASCADE: a kapcsolódó gyermekrekordokat is törli. ON DELETE SET NULL: a hivatkozást NULL-ra állítja, ha az oszlop ezt engedi. CASCADE csak tudatos üzleti döntéssel legyen használva.

## 8. Tranzakciók és konkurens módosítások

```
BEGIN;
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;
UPDATE accounts SET balance = balance + 1000 WHERE id = 2;
COMMIT;
-- Hiba esetén: ROLLBACK;
```

ACID: atomicitás, konzisztencia, izoláció, tartósság. A tranzakció garantálja, hogy a két művelet együtt commitolódik vagy rollbackelődik; önmagában nem pótolja a helyes üzleti validációt.

```
BEGIN;
SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;
-- üzleti ellenőrzés és módosítás ugyanabban a tranzakcióban
COMMIT;
```

A SELECT ... FOR UPDATE sorzárat kér, így segíthet elkerülni az elveszett módosításokat. Ismerd a deadlock fogalmát és a tranzakciók röviden tartásának jelentőségét. PostgreSQL alapértelmezett izolációja READ COMMITTED.

## 9. Indexek és alap teljesítmény

```
CREATE INDEX ix_orders_user_id ON orders(user_id);
CREATE INDEX ix_orders_status_created
ON orders(status, created_at DESC);

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders
WHERE user_id = 1
ORDER BY created_at DESC;
```

Az index felgyorsíthatja a WHERE/JOIN/ORDER BY műveleteket, de tárhelyet és írási többletköltséget okoz. A FOREIGN KEY oszlophoz PostgreSQL nem hoz létre automatikusan indexet. Összetett indexnél az oszlopsorrend fontos. Az EXPLAIN ANALYZE ténylegesen végrehajtja a lekérdezést; módosító parancsoknál óvatosan használd.

Ne feltételezd, hogy minden Seq Scan hibás: kis táblán sokszor ez a legjobb terv. Kerüld a SELECT *-ot széles táblákon és API-kban, ha csak néhány oszlop kell. Figyelj az N+1 lekérdezési problémára.

## 10. SQL biztonság: paraméterezés

Felhasználói inputot ne fűzz SQL stringbe. Paraméterezett lekérdezéssel az adat érték marad, nem SQL kód; ez az SQL injection elleni alapvédelem.

```
// Npgsql példa
await using var cmd = new NpgsqlCommand(
    "SELECT id, name FROM users WHERE email = @email", connection);
cmd.Parameters.AddWithValue("email", email);
await using var reader = await cmd.ExecuteReaderAsync();
```

Dinamikus táblanév vagy ORDER BY oszlopnév nem paraméterezhető ugyanígy: használj explicit engedélyezett értéklistát (allowlist). Az alkalmazás adatbázis-felhasználójának csak a szükséges jogosultságokat add.

## 11. Entity Framework Core és LINQ megfeleltetések

```
// SQL: SELECT * FROM orders WHERE total > 5000
var orders = await db.Orders
    .AsNoTracking()
    .Where(o => o.Total > 5000)
    .ToListAsync();

// SQL: GROUP BY user_id, COUNT, SUM
var stats = await db.Orders
    .GroupBy(o => o.UserId)
    .Select(g => new {
        UserId = g.Key,
        Count = g.Count(),
        Total = g.Sum(x => x.Total)
    })
    .ToListAsync();
```

A LINQ-kifejezést az EF Core provider SQL-re fordítja, amennyiben támogatott. A lekérdezés tipikusan a ToListAsync/FirstOrDefaultAsync/CountAsync hívásnál fut le (deferred execution).

```
// Projekció API DTO-ra: csak szükséges oszlopok
var result = await db.Users
    .AsNoTracking()
```

```
    .Select(u => new {
        u.Id, u.Name,
        OrderCount = u.Orders.Count()
    })
    .ToListAsync();

// Lekérdezés vizsgálata
var sql = db.Orders.Where(o => o.Total > 5000)
    .ToQueryString();
```

Include akkor hasznos, ha entitások kapcsolódó adatait is betöltöd. DTO-projekciónál gyakran nem kell Include. Figyeld a generált SQL-t, a roundtrip-ek számát és a lekért adatmennyiséget.

### EF Core migrációk

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

A migráció a modell és adatbázisséma változásait verziózza. Éles környezetben a migrációkat ellenőrzött telepítési folyamattal alkalmazd; ne vakon automatikusan minden induláskor.

## 12. Tipikus junior hibák – ellenőrzőlista

• JOIN után váratlanul több sor → 1:N kapcsolat; ellenőrizd a kardinalitást.

• LEFT JOIN + WHERE a jobb oldalon → eltűnhetnek a pár nélküli sorok.

• COUNT(*) vs COUNT(nullable_column) → eltérő eredmény.

• NULL = NULL nem TRUE; használj IS NULL-t.

• SUM() üres eredménynél NULL; COALESCE szükséges lehet.

• GROUP BY mellett a nem aggregált SELECT oszlopok csoportosítását ellenőrizd.

• UPDATE/DELETE WHERE nélkül → tömeges módosítás.

• Pénzhez floating point helyett NUMERIC/decimal.

• EF Core-ban ToList túl korán → memóriában végzett szűrés és nagy adatforgalom.

• N+1 lekérdezések, hiányzó indexek, túl sok oszlop lekérése.

## 13. Önálló gyakorlófeladatok

1. Kérd le az összes fizetett rendelést összeg szerint csökkenő sorrendben.

2. Listázd a felhasználókat, akiknek nincs rendelésük.

3. Számold meg felhasználónként a rendeléseket, a nulla rendelést is beleértve.

4. Listázd azokat a felhasználókat, akik legalább 10 000 Ft-ot költöttek.

5. Számítsd ki a fizetett rendelések átlagos értékét.

6. Keress duplikált email címeket egy olyan táblában, ahol nincs UNIQUE constraint.

7. Adj új rendelést egy létező felhasználóhoz és kérd vissza az új ID-t.

8. Készíts indexet a rendelések user_id oszlopára.

9. Írj LINQ lekérdezést a 5000 Ft feletti rendelésekhez.

10. Magyarázd el, miért lehet hibás COUNT(*) LEFT JOIN esetén.

# 14. Megoldókulcs

```
-- 1
SELECT * FROM orders WHERE status = 'paid' ORDER BY total DESC;
-- 2
SELECT u.* FROM users u
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id=u.id);
-- 3
SELECT u.id, COUNT(o.id) FROM users u
LEFT JOIN orders o ON o.user_id=u.id GROUP BY u.id;
-- 4
SELECT u.id, u.name FROM users u
JOIN orders o ON o.user_id=u.id
GROUP BY u.id, u.name HAVING SUM(o.total)>=10000;
-- 5
SELECT AVG(total) FROM orders WHERE status='paid';
-- 6
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*)>1;
-- 7
INSERT INTO orders(user_id,total) VALUES(1,4500) RETURNING id;
-- 8
CREATE INDEX ix_orders_user_id ON orders(user_id);
-- 9 (C#)
// await db.Orders.Where(o => o.Total > 5000).ToListAsync();
-- 10: A LEFT JOIN rendelés nélküli userhez is létrehoz
-- egy NULL-os sort; COUNT(*) ezt is beleszámolja.
```

# 15. Továbblépési terv

1. hét: SELECT, WHERE, rendezés, NULL, INSERT/UPDATE/DELETE. 2. hét: JOIN, GROUP BY, HAVING, EXISTS. 3. hét: constraint-ek, indexek, EXPLAIN, tranzakciók. 4. hét: ugyanazok a lekérdezések EF Core LINQ-ban, ToQueryString ellenőrzéssel, API endpointokba integrálva.

Következő témák: ablakfüggvények (ROW_NUMBER, SUM OVER), keyset pagination, isolation level-ek, query plan elemzés, optimista konkurenciakezelés, összetett indexek és SQL migrációs stratégia.

Tanulási tanács: minden lekérdezést előbb írj meg kézzel SQL-ben (pl. pgAdmin vagy psql), utána LINQ-ban; végül hasonlítsd össze a generált SQL-t és az eredményt.